Proyecto Compras (Flutter)

Documentación técnica: arquitectura, componentes y buenas prácticas

---

1. ¿Qué función cumple main.dart?

El archivo **lib/main.dart** es el **punto de entrada** de la aplicación Flutter. Contiene la función **main()**, que se ejecuta al iniciar la app, y la cual llama a **runApp()** para montar el árbol de widgets en pantalla.

En este proyecto, **main()** instancia **MyApp**, un StatelessWidget que devuelve un **MaterialApp** con las siguientes configuraciones:

- **debugShowCheckedModeBanner: false** → oculta la banda de depuración.
- **title: 'Compras'** → título de la aplicación.
- **theme** → tema visual basado en un color semilla azul (ColorScheme.fromSeed).
- **home: HomePage()** → define la pantalla inicial del formulario de registro de productos.

En resumen: main.dart **arranca la aplicación, configura el tema y la pantalla principal**.

2. ¿Qué función cumple un Widget personalizado?

Un Widget personalizado es un componente reutilizable creado por el desarrollador que empaqueta un diseño o comportamiento propio, evitando repetir código en varias pantallas.

En este proyecto existen tres widgets personalizados en **lib/widgets/**:

- **CustomButton.dart** → envuelve un ElevatedButton con texto (placeholder) y un evento (onClick). Se usa para «Calcular» y «Regresar».
- **CustomTextField.dart** → envuelve un TextFormField con teclado según el tipo de dato, validador y sufijo «%» opcional para descuentos.
- **CustomContainer.dart** → contenedor estilizado con bordes redondeados, sombra, color o degradado, usado para dar formato a las tarjetas de las pantallas.

Ventajas: **reutilización, coherencia visual y mantenimiento centralizado** (si cambia el estilo, se modifica un solo archivo).

3. ¿Por qué se creó compra_service.dart?

El archivo **lib/services/compra_service.dart** se creó para **separar la lógica de negocio de la interfaz de usuario**, aplicando el principio de responsabilidad única.

La clase **CompraService** expone el método **calcularTotal(CompraModel)**, que:

- Calcula el **subtotal** (precio × cantidad).
- Valida y limita el porcentaje de descuento entre 0 y 100 con **clamp(0.0, 100.0)**.

- Calcula el **descuento** y el **total** (subtotal – descuento).
- Redondea los montos a 2 decimales.
- Devuelve un objeto **ResultadoCompra** con toda la información calculada.

Beneficios: la pantalla **HomePage** solo recolecta los datos y muestra resultados, mientras que el cálculo vive en un servicio independiente que puede **probarse por separado y reutilizarse** si en el futuro se conecta a una API o base de datos.

4. ¿Qué información almacena compra_model.dart?

El archivo **lib/model/compra_model.dart** define las clases de datos (modelos) inmutables de la aplicación:

CompraModel – representa una compra registrada por el usuario:

- **nombreProducto** (String): nombre del producto.
- **precio** (double): precio unitario.
- **cantidad** (double): unidades a comprar.
- **porcentajeDescuento** (double): descuento aplicado (0–100).

ResultadoCompra – representa el resultado del cálculo:

- **compra**: la compra original asociada.
- **subtotal**, **descuento** y **total** (double): montos calculados.

Ambas clases usan **const** y campos **final**, lo que garantiza que los datos no cambien una vez creados (inmutabilidad).

5. ¿Qué función cumple Navigator?

Navigator es el gestor de rutas (pantallas) de Flutter. Se encarga del **apilado (stack) de pantallas** y de la navegación entre ellas.

En el proyecto se usa de dos formas:

- **Navigator.push(...)** en **HomaPage.dart**: al pulsar «Calcular», se abre la pantalla **ResultadoScreen** mediante MaterialPageRoute, apilándola sobre la pantalla actual.
- **Navigator.pop(context)** en **ResultadoScreen.dart**: el botón «Regresar» cierra la pantalla de resultado y vuelve a la anterior (HomePage).

Además, permite pasar datos entre pantallas: en este caso se envía el objeto **ResultadoCompra** como parámetro al construir ResultadoScreen.

6. ¿Por qué se utiliza dispose()?

dispose() es el método del ciclo de vida de un StatefulWidget que se ejecuta cuando el widget es **eliminado del árbol de widgets de forma permanente**. Su propósito es **liberar recursos** para evitar fugas de memoria.

En **HomaPage.dart**, el estado (_HomePageState) crea cuatro **TextEditingController** (cntNombre, precio, cantidad y descuento). Al finalizar el widget se hace:

```
cntNombre.dispose(); precio.dispose(); cantidad.dispose(); descuento.dispose();
```

Si no se liberaran, los controladores seguirían ocupando memoria aunque la pantalla ya no exista. Llamar a **super.dispose()** al final es obligatorio para que Flutter complete la limpieza del propio State.